Building a Prisma ORM Backend for PostgreSQL

Overview

This practical exercise will guide you through creating a new Node.js backend application using Prisma ORM to connect to your existing PostgreSQL database. By building a parallel implementation, you'll be able to compare the ORM approach with the direct connection approach from the previous practical.

Part 1: Creating a New Prisma Project

1.1 Set Up a New Project Directory

Create a new project directory separate from your previous one:

```
mkdir prisma-connection
cd prisma-connection
npm init -y
```

1.2 Install Required Dependencies

Install Express, CORS, and Prisma dependencies:

```
npm install express cors
npm install prisma --save-dev
npm install @prisma/client
```

1.3 Initialize Prisma

Initialize Prisma in your project:

```
npx prisma init
```

This creates:
A prisma directory with a schema.prisma file
A .env file for environment variables

1.4 Configure Database Connection

Update the .env file with your PostgreSQL connection details to connect to your existing database:

```
DATABASE_URL="postgresql://username:password@localhost:5432/studentdb?schema=public"
```

Note: Replace username, password, and database name with your actual PostgreSQL credentials. This should be the same database you used in the previous practical.

Part 2: Creating the Prisma Schema

2.1 Introspect the Existing Database

Since you already have a database with the "students" table from the previous practical, use Prisma's introspection feature to generate the schema automatically:

```
npx prisma db pull
```

This command will examine your database structure and generate the corresponding Prisma schema in `schema.prisma`.

2.2 Review and Adjust the Generated Schema

Open the `schema.prisma` file and check the generated model. It should look something like this:

```
model students {
        id              Int              @id @default(autoincrement())
        name            String?
        age              Int?
        grade           String?
        major           String?
}
```

You might want to adjust the schema for better Prisma conventions. For example:

```
model Student {
        id              Int              @id @default(autoincrement())
        name            String?
        age              Int?
        grade           String?
        major           String?

}
```

2.3 Generate the Prisma Client

Generate the Prisma Client based on your schema:

```
npx prisma generate
```

This creates type-safe database client code tailored to your schema.

Part 3: Building the Express Server with Prisma

3.1 Create a Prisma Client Instance

Create a new file named `prisma.js` to configure and export the Prisma Client:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

module.exports = prisma;
```

3.2 Create the Server File

Create a new `server.js` file for your Express application:

```
const express = require('express');
const cors = require('cors');
const prisma = require('./prisma');

const app = express();
const port = 5100; // Using a different port from the original app

// Middleware
app.use(cors());
app.use(express.json());

```

```
// Routes
app.get('/students', async (req, res) => {
  try {
    // Using Prisma to fetch all students
    const students = await prisma.students.findMany();
    res.json(students);
  } catch (error) {
    console.error('Error fetching students:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});

app.get('/students/:id', async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    // Using Prisma to fetch a student by ID
    const student = await prisma.students.findUnique({
      where: { id }
    });

    if (!student) {
      return res.status(404).json({ error: 'Student not found' });
    }

    res.json(student);
  } catch (error) {
    console.error('Error fetching student:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});
```

```
app.post('/students', async (req, res) => {
  try {
    const { name, email, course, enrollment_date } = req.body;

    // Using Prisma to create a new student
    const newStudent = await prisma.students.create({
      data: {
        name,
        email,
        course,
        enrollment_date: new Date(enrollment_date)
      }
    });

    res.status(201).json(newStudent);
  } catch (error) {
    console.error('Error creating student:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});

```

```
// Gracefully shut down Prisma when the app terminates
process.on('SIGINT', async () => {
  await prisma.$disconnect();
  process.exit();
});

// Start the server
app.listen(port, () => {
  console.log(`Prisma server running on http://localhost:${port}`);
});
```

Part 4: Testing the Prisma-Powered API

4.1 Test using a Web Browser

```
http://localhost:5100/students
```

4.2 Test using cURL